ASTRA

How ASTRA watches AI in production

AI models: the monitor, the key, the feed and the thresholds. AI agents: events and OpenTelemetry telemetry.

Explainer based on the current code and API · Version 1.0 · 22 October 2026

1. The idea in one page

A design is a promise. It says what an AI system is meant to do and what risk it carries. Production is where the promise is tested: a model's data drifts, a provider swaps the model behind its API, staff stop checking the outputs, an agent starts calling a tool nobody approved. ASTRA therefore has a second half that **watches the running system and compares it with what was approved**.

The two kinds of system need different evidence, so they are watched differently:

	AI models	AI agents
What is being judged	How the model's inputs and outputs **behave** compared with what was normal.	What the agent **does** (tools, servers, handoffs, data, memory) compared with its **ratified contract**.
What reaches ASTRA	Each prediction: inputs, output, confidence, latency, optional true answer. Only statistics are kept.	Events that carry **names only**: a tool, a server, an agent, a data store. Never arguments, results or prompts.
How it is sent	The model's application calls ASTRA with a few lines of code (SDK or plain HTTP).	The agent's SDK, or the OpenTelemetry traces the organisation already produces.
The baseline	Learned from the first records (normal behaviour).	The ratified contract (allowed behaviour).
What comes out	Status (stable, watch, drift), incidents, facts in the risk score, alerts, evidence in packs.	Findings, runtime drift items for compliance to decide, discovered agents, coverage.

> **What both have in common**
> Everything arrives through an **API key with the Collector role**, which can send data and nothing else. ASTRA only **advises and records**: it never switches a model or agent off. Prompts, replies and records are not stored.

2. AI models

2.1 The four steps, and why each exists



Figure 1. Connecting a model to production.

Step 1: create the monitor (the “Connect production data” form)

The monitor is the **receiving end with a rulebook**. ASTRA never keeps the records, so it must be told up front which statistics to keep for this model. The form is that declaration, a small data contract for the model. You fill it in once:

- **One example record, as JSON.** Its only job is to teach ASTRA the field names and types, so made-up values are enough. Nothing in it is stored. *Suggest the fields* guesses a kind for each input; you correct it (an integer code such as a postcode band is really a category, not a number).
- **Kind of each input:** number, category, yes/no, or text. The kind decides which statistic is kept (see 2.3). For a number you may give a **valid minimum and maximum**; a value outside it is later reported as a domain violation.
- **What the model does:** classifies (a category out), predicts a number, or generates text. This decides how accuracy is judged and, for text models, switches on the extra checks in section 2.5.
- **Output field and favourable outcome.** The favourable outcome (for example “approve”) lets ASTRA compare how often each group receives it.
- **Groups to compare** (for example gender) for the fairness check, and **flags** your code sets (for example a refusal).
- **Records for the baseline** (default 1,000; between 100 and 100,000): how many of the first records count as “normal”.

> **Demo tip**
> The grey example in the box (age, plan, income, gender) is only a placeholder. For a different model paste **that model’s own record**, otherwise *Suggest the fields* will propose the wrong inputs. The Lamad example prints the right record with `python sdk/python/examples/demo_lamad.py health --show-record`.

Step 2: create an API key

The monitor cannot read the model’s traffic on its own: ASTRA is not inside the customer’s system. Something has to **send** the traffic, and the key is what lets it. An administrator creates it under **Users → API keys** with the role **Collector**.

- A Collector key can **send records, decisions and canary results, and nothing else**: it cannot read a monitor, change a setting or sign anything off. If it leaks, the worst outcome is false data being sent, not data being read.
- It is separate from any person’s login, one per system, shown once, and can be revoked without affecting anyone.
- It travels as a bearer token: `Authorization: Bearer <key>`. Roles with the ingest permission are administrator, compliance, engineer and collector; the auditor role cannot send.

Step 3: send the feed

The application that serves the model reports each prediction. With the Python SDK this is a few lines placed where the model answers:

```
from astra_runtime import ModelMonitor

mon = ModelMonitor("lamad-meter-health")   # URL and key from the environment

mon.log(inputs={"meter_type": "coriolis", "dp_noise": 0.4, "alarm_count": 1},
        output="healthy", confidence=0.88, latency_ms=24,
        label=None)                        # the true answer, when it becomes known
```

Any language can do the same with one HTTP request (up to 1,000 records and 2 MB per request):

```
POST /api/model-monitor/lamad-meter-health/records
Authorization: Bearer <collector key>
Content-Type: application/json

{"records": [{"inputs": {"meter_type": "coriolis", "dp_noise": 0.4},
              "output": "healthy", "confidence": 0.88, "latency_ms": 24,
              "model_version": "v3", "at": "2026-10-22T09:15:00Z"}]}
```

The SDK is built so that it can sit in a production path: it **never raises an error into the model and never blocks it**. Records go on a bounded in-memory queue and a background thread sends them in batches, honouring `Retry-After` when ASTRA asks it to slow down. A random `sample` can be set for very high traffic: drift shows up in a sample just as well.

Step 4: ASTRA judges

The first records become the **baseline**. Everything afterwards is compared with it over the last 24 hours. Until the baseline is complete the monitor shows *Learning*; with fewer than 100 records in the window it shows *Too few records* rather than guessing.

2.2 What is sent, and what is kept

ASTRA **receives** the inputs (over HTTPS, as the application sends them) and folds each record into running statistics in memory; the record itself is not stored. Only the fields declared in the form are read; anything else in the record is ignored.

Kind of input	What ASTRA keeps (per hour)
Number	A histogram, minimum and maximum, average and spread, how often it is missing, and how many values fell outside the declared range.
Category	Counts per value, for up to 50 distinct values per window. Anything beyond that is counted as “other”, so an identifier cannot be accumulated.
Yes/no	Counts.
Text	**Only the length distribution. The text is never kept.**
Output, confidence, latency	The same statistics; a confidence histogram; latency buckets; an error count; right and wrong answers once the true answer arrives; the favourable-outcome rate per group.

> **Two details worth knowing**
> **During learning only**, ASTRA holds a random sample of up to 1,000 individual numeric values per number field, with no link to any record or person. When the baseline is taken that sample is deleted and only the histogram remains. **Category values are stored as the labels themselves** (“north”, “coriolis”), so do not send an email address or customer id as a category. Retention is 30 days by default (7 to 90).

2.3 The baseline and the comparison

For every input and for the output, ASTRA measures how far today’s distribution has moved from the baseline using the **Population Stability Index (PSI)**, a standard measure of distribution shift: below 0.10 is stable, 0.10 to 0.25 is worth watching, above 0.25 is drift. It also compares missing-value rates, confidence, error rate and response time, and, once true answers arrive, **accuracy** (classification) or **error size** (regression) against the baseline. A baseline can also be supplied from the training data instead of learned from production.

2.4 The thresholds

A threshold is the point where "a bit different" becomes **watch**, and where it becomes **drift**. They are common rules of thumb, **not calibrated to any one model**, and every one can be changed per monitor.

Threshold	What it means	Watch	Drift
Input and output shift (PSI)	How far the distribution has moved	0.10	0.25
Accuracy drop	Fall once true answers are known (needs 30 labels)	5 points	10 points
Error size (regression)	Error compared with the baseline error	1.25×	1.5×
Error rate	Share of requests that fail	2%	5%
Response time	Median time compared with the baseline	2×	4×
Fairness ratio	Lowest group's favourable rate over the highest (needs 30 per group); the four-fifths rule	below 0.90	below 0.80
Flag rate	A named flag rising compared with the baseline	+5 points	+10 points
Out-of-range values	Values outside the declared minimum or maximum		5 or more
Window and minimum	Records are judged over the last 24 hours; at least 100 are needed		

Who decides. The model owner knows the model and may tune any threshold. But a threshold is also a control, because loosening it makes a problem harder to see. So **compliance sets an organisation-wide floor**: the loosest value any monitor may use. An owner can tighten freely; going looser needs a **compliance sign-off with a written reason**, which is recorded as a waiver and shown on that monitor. If a floor is tightened later, a looser monitor is held to it automatically unless a waiver covers it.

2.5 Beyond distributions: four more signals

Signal	How it works	Needs
Provider changed the model	Each record may carry `model_version` (the model id or fingerprint the provider returns). A new version is a change event, compared with the one before it on answers, errors, response time and confidence. A change with a behaviour shift opens a supply-chain incident.	`model_version` on records
Canary checks	Fixed questions with a known good answer, run on a schedule through the same route as real traffic. A check that passed on earlier versions and fails on the new one shows the model changed, with no baseline needed.	`POST …/canaries` or `mon.run_canary(…)`
What leaves and what comes back unsafe	For text models the SDK scans prompts and replies **inside the customer's own application** for personal data, credentials and prompt-injection signs. Only yes/no flags are sent, never the text. One credential is a finding however rare; personal data and injection signs are judged by rate. These are pattern matches, not proof.	`prompt_text=` and `output_text=` on `mon.log`

Signal	How it works	Needs
How people use the outputs	Decision records say what a person did with an output (accepted, edited, overridden, escalated, ignored), by **role**, how long they looked, and whether the model was right. ASTRA reports rubber-stamping and, more seriously, wrong outputs being accepted. Names are dropped unless compliance signs off counting individuals, and then they become one-way pseudonyms.	POST …/decisions or mon.decision(…)

2.6 What ASTRA does with a finding

- **Status** on the monitor: Learning, Stable, Watch or Drift, with each reason in words.
- **Incidents** for serious findings (one per kind while open): drift, bias breach, model failure, supply chain, human review failing, data leaving, unsafe output. They reach the incident log and feed the risk score; a person decides when to close them.
- **Facts in the risk score.** Drift, a bias breach, domain violations and unsupported human review are added to the model's lifecycle facts, shown as "Seen in production". Production can raise a declared risk, never lower it.
- **The assurance gap.** The Overview shows the score on what people declared next to the score once production evidence is added, with the contradictions listed.
- **Alerts.** A new incident can be sent to a webhook (Slack, Teams, Mattermost or any JSON endpoint). The address is stored encrypted and checked against private addresses; the message says what was found and that ASTRA only advises.
- **Evidence.** Each model's monitors are summarised, as statistics only, in its Assurance Pack.

2.7 API reference for models

Paths are under /api/model-monitor, and {key} is the monitor's key (its name in lower case with hyphens), unless shown in full.

Endpoint	Purpose	Who
POST /api/model-monitor	Create a monitor from the form	Engineer, compliance, admin
POST /api/model-monitor/infer-schema	Suggest the fields from one example record	Same
GET /api/model-monitor, GET /{key}	List monitors; one monitor with a fresh evaluation	Anyone signed in
POST /{key}/records	The feed: up to 1,000 records and 2 MB per request; 120,000 records a minute per key	Collector key
POST /{key}/decisions	What people did with outputs (by role)	Collector key
POST /{key}/canaries	Canary check results	Collector key
POST /{key}/baseline/freeze, PUT /{key}/baseline, POST /{key}/baseline/reset	Take the baseline now, supply it from training data, or start again	Write
GET /{key}/series, /versions, /safety, /canaries, /decisions	History for charts, model versions, safety counters, canaries, how people use outputs	Read

Endpoint	Purpose	Who
PATCH /{key}, PUT /{key}/decision-settings	Thresholds (within the floor), design link, counting individuals	Write; waiver needs compliance
GET/PUT /api/model-monitor/floors	The organisation's threshold floors	Read; set needs compliance
GET /api/model-monitor/signals?design_key=	What production shows for one model design, as the risk score understands it	Read
/api/model-monitor/alerts/channels	Alert channels (webhooks) and a test message	Administrator

2.8 Is this valid for small businesses and start-ups?

Yes, with honest conditions.

- **Easy to run:** self-hosted with Docker, no GPU, no language model needed for monitoring, cheap, and nothing sensitive is kept because only statistics are stored.
- **It needs a developer** to add a few lines where the model answers. There is no code-free option yet, and reading inference logs straight from a database or warehouse is not built.
- **Volume matters.** The window is 24 hours and needs 100 records, so a model with very little traffic is judged slowly. The baseline size and the window can be reduced.
- **Rules of thumb, not guarantees.** The thresholds and risk weights are not calibrated against real incidents, and the pack says so. Accounts are local, with one approver and no single sign-on yet.

3. AI agents

3.1 Why agents are watched differently

A model is judged by how its data behaves. An **agent acts**: it calls tools, connects to servers, hands work to other agents, reads data and writes memory. Its design therefore becomes a **contract**, a signed, tamper-evident record of exactly what it may do. Watching an agent means comparing what it does with that contract, and the comparison is **deterministic**: plain checks against the contract, no model involved.



Figure 2. From agent behaviour to a finding.

3.2 Three ways in

Route	Best when	Needs
The runtime SDK (astra_runtime)	You own the agent's code and want precise events, and optionally enforcement.	A few lines, or an adapter for the OpenAI Agents SDK or LangChain/LangGraph.
OpenTelemetry traces	The organisation's agents are already instrumented (OpenInference, OpenLLMetry, the OpenAI Agents SDK, LangChain, LlamaIndex, CrewAI and others).	Point the exporter or collector at ASTRA. No code change.

Route	Best when	Needs
Discovery sources	To find agents that nobody has told ASTRA about.	A connected repository or identity provider (these find agents; the two routes above watch them).

All routes use a Collector API key, and **all routes end in the same place**: the SDK endpoint and the OpenTelemetry receiver both turn what they receive into the same runtime events, which are judged by the same code. So an agent seen by two routes is one inventory entry, judged once.

3.3 The runtime SDK and the events API

An event reports one thing the agent did. The SDK sends them in batches to `POST /api/runtime/events`:

Event type	Fields	Compared with the contract's
`tool_call`	`name`, `write` (does it change anything)	allowed tools, and whether writing is allowed
`mcp_connect`	`name`, `signed`	allowed MCP servers, and whether signed
`delegation`	`name` (the agent handed work to)	allowed agents to hand work to
`data_access`	`name` (a data store)	allowed data stores
`memory_write`	`scope`: long_term or session	whether long-term memory is allowed
`heartbeat`	none	nothing: it only shows a quiet agent is still alive

```
import os, astra_runtime as astra

astra.init(url="https://astra.example.org",
           api_key=os.environ["ASTRA_KEY"], agent="refund-agent")

astra.tool_call("stripe.refunds.create", write=True)
astra.delegation("notification-agent")
astra.mcp_connect("mcp://orders", signed=True)

@astra.tool("lookup_order")            # report every call of a function
def lookup_order(order_id): ...

# or hook a framework:  astra_runtime.adapters.openai_agents.install()
#                       astra_runtime.adapters.langchain.handler()
```

What the SDK sends is **names only**: the kind of event, one name, a time and where given `write`, `signed` or `scope`. Never arguments, results, prompts or data. It never raises into the agent and never blocks it (a queue of 5,000 and a background thread sending batches of up to 50, with back-off), uses only the Python standard library, and sends a heartbeat every 30 seconds.

Optional enforcement

With `enforce=True`, the SDK fetches a summary of what the contract allows (`GET /api/runtime/contract/{agent}`, cached for 60 seconds) and **refuses a wrapped tool call before it runs** if the contract does not allow it. If the agent has no contract or ASTRA cannot be reached it **allows** the call (fail open), so an outage never stops the agent; `strict=True` refuses instead. Enforcement covers only the calls you wrap.

3.4 OpenTelemetry telemetry

Most organisations already emit OpenTelemetry traces: a span for each agent run, tool call and model call. ASTRA reads them over **OTLP/HTTP** (JSON or protobuf, optionally gzip) and turns spans into the same events as above, so an agent can be discovered and checked **without the SDK**.

Pointing an exporter at ASTRA

```
OTEL_EXPORTER_OTLP_TRACES_ENDPOINT=https://astra.example.org/api/telemetry/v1/traces
OTEL_EXPORTER_OTLP_HEADERS=Authorization=Bearer <collector key>
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
```

The same endpoint works from an OpenTelemetry Collector's `otlphttp` exporter. A request is limited to 16 MB (also after decompression) and 20,000 spans, and is rate limited per key; ASTRA answers 429 with `Retry-After`, which OTLP exporters honour.

What is read, and what never is

Convention	What ASTRA takes from it
OpenTelemetry GenAI	`gen_ai.operation.name` (invoke_agent, execute_tool, chat), `gen_ai.agent.name`, `gen_ai.tool.name`, the model name and token counts
OpenInference (Arize Phoenix, OpenAI Agents SDK, LangChain, LlamaIndex, CrewAI)	`openinference.span.kind`: agent, tool or LLM
OpenLLMetry (Traceloop)	`traceloop.span.kind` and the entity name
MCP and databases	`mcp.method.name`, `mcp.server.name`; `db.system` with the database name. For a database span only the **first word** of the query (INSERT, UPDATE, DELETE and similar) is looked at, to mark the access as a write; it is not kept
OpenAI Agents SDK handoffs	the handoff span and the agent handed to

> **Never read, never kept**
> **Prompts, completions, tool arguments and results, message content and span events** are not read or stored. Only names, kinds, timings, the model name and token counts are kept. The one exception in what is *looked at* is the first word of a database query (INSERT, UPDATE and so on) to tell a read from a write; that word is not stored.

Which agent does a span belong to?

1. The span's own `gen_ai.agent.name`.
2. Otherwise the **nearest ancestor span that is an agent span**, so a tool call inside an agent run belongs to that agent even if the batch arrives in pieces.
3. Otherwise, only if the organisation allows it, the **service name**.

A span that cannot be placed is counted, and a few names are kept so the person setting it up can fix the instrumentation, but it creates nothing.

Telemetry API

Endpoint	Purpose
`POST /api/telemetry/v1/traces`	The OTLP/HTTP traces endpoint (Collector key or engineer key)
`GET /api/telemetry/status`	What arrived in the last 24 hours, what could not be understood, and the settings

Endpoint	Purpose
PUT /api/telemetry/settings	Whether the service name may identify an agent (agent_from_service) and whether an unknown agent is registered (discover)
GET /api/telemetry/agents	What each agent was seen doing: tools, MCP servers, delegations, data, models, errors
POST /api/telemetry/agents/{key}/design	A draft design built from what the agent was seen doing (evidence of what happened, not of what could)

3.5 How an event is judged

Every event, from the SDK or from telemetry, goes through ingest_batch. It records the event and produces one of four verdicts:

Verdict	Meaning
CONFORMING	The agent has an active contract and the event is allowed.
DIVERGENT	The event is not allowed by the contract. A finding is raised.
NO_CONTRACT	The agent is known but has no active contract, so it cannot diverge from anything.
UNKNOWN_AGENT	The platform had never heard of this agent. It is registered, **unowned**, with origin DISCOVERED; it counts against ASI10 until someone owns it.

A divergence becomes a finding

Class	Event	OWASP agentic category	Severity
DVG-TOOL	A tool outside the contract	ASI02 Tool misuse and exploitation	High if it writes, else Medium
DVG-SUP	An MCP server outside the contract	ASI04 Agentic supply chain vulnerabilities	Medium
DVG-DEL	A handoff to an unlisted agent	ASI07 Insecure inter-agent communication	Medium
DVG-DATA	A data store outside the contract	ASI03 Identity and privilege abuse	Medium
DVG-MEM	Long-term memory not allowed	ASI06 Memory and context poisoning	Low

The same open finding is not raised again within ten minutes, and each finding carries an evidence hash so it can be verified later. For a **new capability** the contract does not allow, ASTRA also opens a **runtime drift item**: the ratified design plus what was observed, for compliance to **approve** (ratify a contract that includes it) or **decline**. A declined observation never creates another drift item; it keeps raising findings.

Limits that protect the platform

- Up to 200 events per request and 1,200 events a minute per key; over that, 429 with Retry-After.
- An event's own timestamp is believed only within a day of the server's clock. Events are kept for 14 days.

- A Collector key can send events and read the contract summary it needs, and nothing else. Nothing in this path can sign anything off.

3.6 API reference for agents

Endpoint	Purpose	Who
POST /api/runtime/events	Report a batch of one agent's events	Collector key
GET /api/runtime/contract/{agent}	What the active contract allows, for local enforcement	Collector key
GET /api/runtime/overview	Live view: which agents are live, recent events and verdicts	Read
POST /api/telemetry/v1/traces	OpenTelemetry traces in; the same events out	Collector key
GET /api/telemetry/status, /agents; PUT /settings	What arrived, what each agent did, how agents are recognised	Read; settings need write

4. Models and agents side by side

	AI models	AI agents
Question asked	Is it still behaving as it did when validated?	Is it still doing only what it was approved to do?
Evidence	Statistics of inputs, outputs, errors, people and versions	Names of tools, servers, agents, data and memory
Compared with	A learned or supplied baseline, and the thresholds	The ratified contract
Entry routes	SDK or HTTP (/api/model-monitor/{key}/records)	SDK, or OpenTelemetry (/api/telemetry/v1/traces)
Key role	Collector	Collector
Stored	Hourly aggregates (30 days by default)	Event names, kinds, verdicts (14 days)
Result	Status, incidents, score facts, alerts, packs	Findings, drift items, discovered agents, coverage
Stops anything?	No: advice and records only	No, unless the SDK's optional enforcement wraps a call

5. Privacy, security and honest limits

Privacy

- **Models:** the inputs are received and folded into statistics; records are not kept. Text inputs are reduced to a length. Prompts and replies can be scanned inside the customer's own application so the text never leaves it. People's decisions are counted by role, and names are never stored unless compliance signs off, and then only as one-way pseudonyms.
- **Agents:** names only. Prompts, completions, tool arguments and results are not read or stored (the first word of a database query is checked to tell a read from a write, and is not kept).

Security

- A Collector key can only send. Requests are size-limited and rate-limited and answered with `Retry-After`.
- Alert webhook addresses and connection secrets are encrypted at rest; a webhook to a private address is refused unless an administrator allows it.
- Every change by a person is recorded in a tamper-evident audit log.

Limits to state plainly

- ASTRA **advises and records**; it does not stop a model. The only blocking is the agent SDK's optional enforcement, which covers only the calls you wrap.
- Thresholds and risk weights are **judgement values, not calibrated**. A clear result is not a certificate.
- Pattern checks for personal data and credentials are matches, not proof: a match can be a false alarm and a miss is possible.
- Not built yet: reading inference logs from a database or warehouse, attributing an LLM's OpenTelemetry spans to a model monitor, re-alerting while an incident is left unattended, and single sign-on. Accounts are local and there is one approver per action.